

GrabNavi – Real-time assistant for smoother rides & deliveries

Introducing **GrabNavi** an autonomous agentic brain built for last-mile operations.

The Challenges:

The last mile is unpredictable:

- 🚦 Drivers **stuck in traffic**.
- 🍲 Restaurants **overloaded**.
- 📦 **Damaged** or **missing** items.
- ⚠️ **Safety** incidents.
- 😡 **Customers waiting** without answers.

Traditional systems log the issue. Human ops teams scramble to fix it. This causes delays, poor customer experience, and stressed drivers.



The Solution: GrabNavi

Think of it as a **dispatcher + problem solver + communicator** — all in one.

Whenever a disruption occurs, GrabNavi:

1. **Understands the issue** (via natural language + structured inputs).
2. Thinks through **possible resolutions** (chain of thought reasoning).
3. Acts using **connected tools & APIs** (traffic rerouting, refunds, notifications).
4. **Logs & Explains** everything **transparently** (so humans can see its reasoning).

Tech Stack



Google colab link :

GitHub link :

Driver 8080 Port :

Customer Colab Link :

[GrabNavi Colab Link](#)

[GrabNavi GitHub](#)

[Driver Permanent Port Link](#)

[Google Colab Link](#)

GrabNavi – Real-time assistant for smoother rides & deliveries

From Disruption to Resolution: GrabNavi workflow



Disruption Detected

(driver / customer / monitoring system reports an issue)



Incident Detection

(detect_incidents_llm() parses the report. Labels issues [e.g., traffic_obstruction, damaged_item, recipient_no_show, restaurant_overload])



Context Gathering

(check_traffic() [Confirm congestion], contact_recipient() [Check availability], analyze_photos(), calculate_alternative_route())



Evidence Collection

(collect_evidence() [photos, ETAs, driver proof]. handle_overloaded_restaurant_case(). locker_options or alternate merchants.)



Decision & Action

(Agent picks tool(s): calculate_alternative_route(). notify_user(). issue_refund() / exonerate_driver(). handle_overloaded_restaurant_case().)



Multi-Issue Orchestration

(Split & prioritize incidents, Deduplicate notifications, Run agent for each incident.)



Audit & Feedback

(Log to DynamoDB, Save reasoning trace, Escalate risky steps for human approval.)



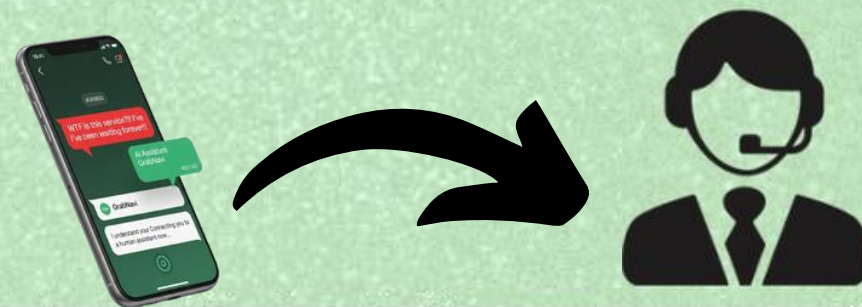
Core Feature - 1 Compound Incident Orchestrator

- **Use case:** Multiple simultaneous issues (e.g., traffic jam + damaged item + recipient no-show) — agent ingests a single report that contains several problems.
- **Why:** It prevents conflicting actions (e.g., reroute vs. return), avoids spamming customers with repeated messages, and shortens time-to-resolution so drivers spend less idle time.
- **How:** LLM extracts incidents → sort by priority → run agent per incident (tool-calls: check_traffic, collect_evidence, contact_recipient, etc.) → dedupe identical notifications → persist human-readable trace and summary; risky actions pause for human approval.



Core Feature - 2 Human-In-Loop Orchestrator

- **Use case:** Human-in-the-Loop Orchestrator — the agent runs routine actions autonomously but pauses for human approval at risky steps.
- **Why:** Some actions carry financial, legal, or brand risks (e.g., issuing large refunds, blacklisting merchants, discarding undelivered food). Fully automating these steps can lead to costly mistakes or poor customer experiences if not reviewed.
- **How:** Agent runs → safe tools automatically → flags risky steps → surfaces to humans → gets approval/override → resumes execution.



Core Feature - 3 Map-Aware Orchestrator

- **Use case:** The agent leverages live map data (traffic updates, optimal routes, delivery zones) to make real-time decisions for drivers and delivery agents.
- **Why:** It reduces delays by avoiding congested routes, ensures accurate ETAs for customers, and helps drivers complete more deliveries with less idle time. It also prevents errors like assigning deliveries outside the coverage area.
- **How:** Agent ingests live map + traffic feeds → checks route feasibility → reroutes dynamically if delays detected → updates ETA and customer notifications → escalates to human if no feasible route exists.



Core Feature - 4 Email - Orchestrator

- **Use case:** The agent generates customer-facing emails on behalf of drivers, summarizing delivery incidents and resolution progress in a single clear message.
- **Why:** It prevents multiple confusing notifications, keeps customers informed with accurate updates, and builds trust by showing proactive communication from the delivery team.
- **How:** Agent collects driver response + incident details → dedupes repeated info → generates concise, human-readable email → sends structured update to the customer → escalates to human support if sensitive content is detected.



Brain of GrabNavi - The Agent

Let's understand The Agent of GrabNavi with a usecase

Assume that driver reported through a message - “Stuck in traffic on Airport Road, ETA keeps increasing”

1) Problem detection (fast path)

Functions: **detect_problems** → if inconclusive **llm_detect_problems** / **detect_incidents_llm**

Purpose: decide this is a **traffic issue** (not damage / payment / missing driver).

Example rule-match: **keywords** = ["traffic", "stuck", "road", "delay", "ETA"] → match → problem = "traffic"

If rules are unsure: **LLM is called** via **make_agent** + **detect_incidents_llm** to return {"type": "traffic", "severity": "moderate", "entities": ["Airport Road"]}

Decision: **route to traffic handler.**

2) Compound check

Functions: **is_compound** / **llm_compound_check**

Purpose: **Ensure the message doesn't hide additional issues** (e.g., also "item damaged").

Outcome here: **False** → single-issue traffic case.

True → compound-issue traffic case.

7) Notifications: tool_notify_user / tool_notify_customer

Inputs: Driver notification payload: ("driver", "Rerouting to avoid traffic on Airport Road. New ETA ~12 min")

Customer notification (via **tool_notify_customer**):

("customer@example.com", "Your driver was delayed due to heavy traffic on Airport Road. We rerouted to save - 10 min. New ETA 12 min. Sorry for the delay")

Simulated outputs: acknowledgement objects; logs appended.

Also: **send_driver_delay_and_log** (using Sendgrid) is used to create an email and append to email_log.json.

3) Orchestrator: run traffic handler

Functions: **run_case_with_trace** → **handle_traffic_case** (or **_handler_traffic**) wrapped with **_wrap_notify**

Purpose: **Central orchestration** that **logs each step** and **ensures notifications.**

High-level sub-steps inside **handle_traffic_case**:

1. call **tool_check_traffic**(order_id)
2. evaluate result; if **incident confirmed**, compute alternative with **tool_calculate_alternative_route**(start, end, CITY_GRAPH)
3. if **reroute viable**, call **tool_re_route_driver**(driver_id, new_route)
4. notify driver & customer via **tool_notify_user** / **tool_notify_customer** and optionally **send_driver_delay_and_log** / **driver_update_via_agent**
5. **append trace** and **produce final summary**

await handle_text_case("Traffic jam at airport road. Driver stuck in jam.")

```
'traces': [{ 'context': { 'end': 'Airport', 'location': 'MainRoad', 'start': 'Restaurant' },
  'label': 'Traffic Obstruction',
  'resolution': 'Traffic Obstruction handled in 3 steps',
  'steps': [{ 'args': { 'location': 'MainRoad' },
    'incident': 'Traffic Obstruction',
    'observation': '{\"location\": \"MainRoad\", \"incident\": \"accident\", \"delay_minutes\": 18}',
    'step': 1,
    'tool': 'check_traffic',
    'ts': '2025-09-09T11:15:15.729481Z'},
    { 'args': { 'end': 'Airport', 'start': 'Restaurant' },
    'incident': 'Traffic Obstruction',
    'observation': '{\"start\": \"Restaurant\", \"end\": \"Airport\", \"path\": [\"Restaurant\", '
      '\"MainRoad\", \"Airport\"], \"eta_minutes\": 26}',
    'step': 2,
    'tool': 'calculate_alternative_route',
    'ts': '2025-09-09T11:15:15.729510Z'},
    { 'args': { 'message': 'Traffic incident detected at MainRoad. Rerouting...',
    'role': 'customer'},
    'incident': 'Traffic Obstruction',
    'observation': '{\"role\": \"customer\", \"message\": \"Traffic incident detected at MainRoad. '
      'Rerouting...\"}',
    'step': 3,
    'tool': 'notify_user',
    'ts': '2025-09-09T11:15:15.729518Z'} ]}] ] }
```

4) Tool call: tool_check_traffic

Input: order_id="ORD-20250908-0001", location="Airport Road"

Decision logic: severity >= moderate and expected_delay > threshold (e.g., 10 min) → **attempt reroute.**

Simulated

output

```
{
  "order_id": "ORD-20250908-0001",
  "incident_type": "heavy_traffic",
  "severity": "moderate",
  "expected_delay_min": 22,
  "blocking_road": "Airport Road",
  "timestamp": "2025-09-08T18:12:00+05:30"
}
```

5) Tool call: tool_calculate_alternative_route

Input: start="Current driver location", end="Customer dropoff", CITY_GRAPH=graph

```
{
  "route": ["Airport Road → Bypass Ave → Highway 2 → Dropoff St"],
  "eta_min": 12,
  "distance_km": 5.2,
  "score": "good"
}
```

6) Tool call: tool_re_route_driver

Input: driver_id="DR-555", new_route=[..]

```
{
  "driver_id": "DR-555",
  "assigned_route": "route_id_987",
  "status": "ok",
  "timestamp": "2025-09-08T18:14:30+05:30"
}
```

8) Agent microservice update: driver_update_via_agent / call_agent_sync

Purpose: persist the new status to the agent/microservice so other systems (support dashboard, tracking) see the change.

Input: payload with **order_id**, **driver_id**, **new_eta**, **route_id**, **trace_id**

Output: {"ok": true, "updated": true, "agent_response": "status updated"}

9) Logging & tracing: run_case_with_trace / _summarize_single_trace

Purpose: capture a **step-by-step trace** of what happened (for audit & explainability).

Trace entries: each call above is **appended to the trace** with timestamp, function name, inputs, outputs.

Final summary (generated by **_single_summary**)

Clients of GrabNavi - Driver & Customer

Now let's see how the agent plans & resolves last mile disruption scenarios

1) Stuck in Traffic (Driver's Issue)

- Detect: **_normalize_issue_text** → **detect_problems** (or **detect_incidents_llm** if ambiguous).
- Plan: confirm delay → try **reroute** if gain $\geq$ threshold.
- Tools used: **tool_check_traffic** → **tool_calculate_alternative_route** → **tool_re_route_driver**.
- Notify & persist: **tool_notify_customer** / **tool_notify_user** + **driver_update_via_agent**.
- Fallbacks: **no viable route** → notify customer, offer cancel/voucher, escalate to **escalate_to_human_agent**.
- Why efficient: rule-first detection (**cheap**), LLM fallback (**accurate**), modular tools = **fast automated fix + audit trace**.

2) Damaged Parcel (Customer's Issue)

- Detect: **_detect_problems** → if unclear, **detect_incidents_llm**.
- Plan: collect evidence → analyze → **decide refund or escalate**.
- Tools used: **tool_collect_evidence** → **tool_analyze_evidence** → **tool_issue_refund** / **tool_exonerate_driver**.
- Decision thresholds: **auto-refund** if **confidence** $\geq$ **threshold** and amount $\leq$ limit; **else escalate**.
- Fallbacks: **missing photos** → **request customer evidence** → **escalate** after TTL.
- Why efficient: **automates routine refunds**, preserves **human review** for risky/ambiguous cases, **full trace for audits**.

3) Overcrowded restaurant (Merchant's Issue)

- Detect: merchant/customer message → **detect_problems** / **detect_incidents_llm**.
- Plan: verify merchant status → propose alternatives or compensation.
- Tools used: **tool_get_merchant_status** → **tool_get_nearby_merchants** → **tool_notify_customer**.
- Decision thresholds: if **delay** $>$ **30 min** → **offer alternatives** → **voucher** or **cancel/refund**.
- Fallbacks: **no alternatives** → larger compensation or **human escalation**.
- Why efficient: proactive notifications **reduce cancellations** & support load; **quick alternatives** and thus **improve CX**.

Backend of GrabNavi - AWS

GrabNavi runs a compact backend stack on an **Amazon Linux EC2 instance** with an **Elastic IP**

- Two Flask services run as permanent processes: the **combined support flask app** on **port 5000** and the **driver's dashboard** on **port 8080**.
- The EC2 instance assumes an **AWS IAM role** with a **custom policy** that permits **AWS Location Service** calls.
- Persistence uses six **DynamoDB** tables. **SendGrid** is used for branded **transactional email**; email metadata is logged.

Core components:

- **EC2 (Elastic IP)**: hosts Flask (ports 5000 & 8080).
- **DynamoDB (6 tables)**: Orders, Drivers, Customers, GrabUpdates, EmailLog, Traces.
- **AWS Location**: traffic and routing (access via EC2 IAM role / custom policy).
- **SendGrid**: transactional email; responses logged.
- **Orchestration**: Flask handlers run detection (rules + LLM fallback), call routing/evidence tools, write DB, send email, and record traces.

Typical event flow – Driver stuck on Airport Road

- **Driver posts** {order_id, driver_id, message, lat,lng} to **POST /driver/update (8080)**.
- Update is **written to** GrabUpdates (DynamoDB).
- Orchestration **normalizes text** and **detects traffic** (rule or **detect_incidents_llm**).
- **AWS Location** is queried (via IAM role) to **confirm congestion** and **estimate delay**.
- If **reroute threshold met**, **tool_calculate_alternative_route** runs;
tool_re_route_driver updates Orders/Drivers **with new route** and **ETA**.
- Customer is notified via **SendGrid**; send status and metadata are **written to EmailLog**.
- **All steps are appended to Traces** for audit; optional agent/**dashboard sync** occurs.

Key Highlights

- **Resilient Infra**: EC2 (Amazon Linux) behind ALB with TLS; **IAM role** (custom policy) for **AWS Location**; **DynamoDB** with 6 tables + **PITR backups**.
- **Smart Orchestration**: **Rule + LLM detection**, **reroute logic** (>10m delay & ≥5m gain), **automated refunds** with **confidence checks**, **SendGrid** for **email logs**.
- **End-to-End Traceability**: Every **update**, **reroute**, and **notification** stored in Traces table for audits, replay, and agent dashboards.